



Instituto **Tecnológico**[®]
de Aguascalientes



INSTITUTO TECNOLÓGICO DE AGUASCALIENTES

Integrante:

Manuel Humberto Reyes Salas

María José Guerrero Ríos

Jesús Eli Sánchez Ruvalcaba

Actividad:

Materia: Tecnologías inalámbricas

Docente: RICARDO ALEJANDRO RODRIGUEZ
JIMENEZ

Fecha: 05 de Junio del 2026

Introducción

El presente proyecto tiene como objetivo simular el funcionamiento de un centro ARCADE utilizando tecnología RFID/NFC, comunicación WiFi mediante ESP32 y una base de datos SQL Server.

El sistema permite identificar jugadores mediante tarjetas RFID/NFC, almacenar sus puntuaciones de forma acumulativa y sincronizar la información a través de una red local utilizando un servidor desarrollado en Node.js.

El proyecto integra tecnologías de hardware y software para demostrar la comunicación entre dispositivos IoT, servidores de aplicaciones y sistemas gestores de bases de datos.

Objetivo General

Desarrollar un videojuego capaz de registrar y acumular puntuaciones de jugadores utilizando tarjetas RFID/NFC y una base de datos centralizada.

Objetivos Específicos

- Leer identificadores únicos mediante RFID/NFC.
- Conectar la ESP32 a una red WiFi.
- Enviar información mediante peticiones HTTP.
- Utilizar un servidor Node.js como intermediario.
- Almacenar información en SQL Server.
- Registrar automáticamente jugadores nuevos.
- Actualizar puntuaciones acumuladas.

Arquitectura General del Sistema

Flujo general:

1. El jugador acerca una tarjeta RFID/NFC.
2. La ESP32 obtiene el UID de la tarjeta.
3. El videojuego inicia.
4. El jugador obtiene una puntuación.
5. La ESP32 envía:
 - UID RFID

- Puntuación obtenida
- 6. Node.js recibe la información.
- 7. SQL Server verifica si el jugador existe.
- 8. Si existe:
 - Actualiza puntuación acumulada.
- 9. Si no existe:
 - Crea un nuevo registro.
- 10. Se almacena la fecha de la última partida.

Tecnologías Utilizadas

Hardware

- ESP32
- Módulo NFC PN532
- Pantalla OLED SSD1306
- Botones físicos
- Cableado para simulación de bomba
- Tarjetas RFID/NFC

Software

- Arduino IDE
- Node.js
- SQL Server
- Docker
- Visual Studio Code

Base de Datos

Base de datos utilizada:

arcade_db

Tabla principal:

Puntuaciones

Campos utilizados:

- ID
- NAME_DISP
- USR
- SCORE
- ID_RFID
- LAST_GAME

Descripción:

Campo	Descripción
ID	Identificador único
NAME_DISP	Nombre del dispositivo
USR	Nombre del jugador
SCORE	Puntuación acumulada
ID_RFID	UID de la tarjeta
LAST_GAME	Última fecha de juego

Creacion de la base de datos y de la tabla con sus campos:

```
C: > Users > mr411 > Downloads > CREATE DATABASE arcade_db;.sql
1  CREATE DATABASE arcade_db;
2  GO
3
4  USE arcade_db;
5
6  CREATE TABLE puntuaciones (
7      ID INT PRIMARY KEY IDENTITY(1,1),
8      NAME_DISP VARCHAR(50),
9      USR VARCHAR(50),
10     SCORE INT,
11     LAST_GAME DATETIME DEFAULT GETDATE(),
12     ID_RFID VARCHAR(50)
13 );
14
15 select * from puntuaciones;
16
17
```

Funcionamiento del Videojuego

El videojuego consiste en una simulación de desactivación de bomba, se desconectarán los cables y si aciertas te dará puntos, pero si te equivocas pierdes.

- Nivel 1

Solo empieza funcionando con 2 cables

Valor base: 50 puntos

- Nivel 2

Se incrementa un cable y ahora son 3 cables

Valor base: 100 puntos

- Nivel 3

Se incrementa un cable y ahora son 4 cables

Valor base: 150 puntos

Si el jugador falla:

El puntaje que se guardara, dependerá del nivel en el que se quedó, pero nota si pierde en el nivel 1 entonces se guarda 0 puntos

Al finalizar:

Se muestran los puntos obtenidos.

Se envían al servidor.

Código del Json

```
// Servidor REST intermediario para ESP32 -> MSSQL
```

```
// npm install express sql mssql cors body-parser
```

```
const express = require('express');
```

```
const sql = require('mssql');
```

```
const cors = require('cors');
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
// Configuración de MSSQL
```

```
const config = {
```

```
  server: '192.168.0.104',
```

```
  port: 1433,
```

```
  user: 'sa',
```

```
  password: 'C0NTR453N1!4',
```

```
  database: 'arcade_db',
```

```
  authentication: {
```

```
    type: 'default',
```

```
    options: {
```

```
      userName: 'sa',
```

```
      password: 'C0NTR453N1!4'
```

```
    }
```

```
  },
```

```
  options: {
```

```
    encrypt: false,
```

```
    trustServerCertificate: true
```

```
  }
```

```
};
```

```
// Ruta para hacer INSERT

app.post('/api/score', async (req, res) => {

  try {

    const { score = 0, id_rfid } = req.body;

    if (!id_rfid) {

      return res.status(400).json({

        success: false,

        message: 'Falta id_rfid'

      });

    }

    const pool = await sql.connect(config);

    // Buscar si existe la tarjeta

    const existe = await pool.request()

      .input('id_rfid', sql.VarChar(50), id_rfid)

      .query(`

        SELECT ID, SCORE

        FROM puntuaciones

        WHERE ID_RFID = @id_rfid

      `);
```

```
// SI EXISTE → UPDATE

if (existe.recordset.length > 0) {

    const scoreActual = existe.recordset[0].SCORE || 0;
    const nuevoScore = scoreActual + (score || 0);

    await pool.request()
        .input('nuevoScore', sql.Int, nuevoScore)
        .input('id_rfid', sql.VarChar(50), id_rfid)
        .query(`
            UPDATE puntuaciones
            SET SCORE = @nuevoScore,
                LAST_GAME = GETDATE()
            WHERE ID_RFID = @id_rfid
        `);

    await pool.close();

    return res.json({
        success: true,
        action: 'UPDATE',
        scoreAnterior: scoreActual,
        scoreGanado: score,
        scoreNuevo: nuevoScore
    });
}
```



```
}
```

```
// NO EXISTE → INSERT
```

```
await pool.request()
```

```
.input('score', sql.Int, score || 0)
```

```
.input('id_rfid', sql.VarChar(50), id_rfid)
```

```
.query(`
```

```
INSERT INTO puntuaciones
```

```
(
```

```
NAME_DISP,
```

```
USR,
```

```
SCORE,
```

```
ID_RFID
```

```
)
```

```
VALUES
```

```
(
```

```
'Inalambricas',
```

```
'Jugador',
```

```
@score,
```

```
@id_rfid
```

```
)
```

```
`);
```

```
await pool.close();
```

```
return res.json({
```

```

    success: true,

    action: 'INSERT',

    score: score,

    id_rfid: id_rfid
  });

} catch (err) {

  console.error(err);

  return res.status(500).json({

    success: false,

    error: err.message

  });

}

});

// Ruta de prueba de conexión
app.get('/api/test', async (req, res) => {
  try {
    let pool = await sql.connect(config);
    await pool.close();

    res.json({
      success: true,

```

```

        message: 'Conexión a MSSQL exitosa'
    });
} catch (err) {
    res.status(500).json({
        success: false,
        message: 'Error de conexión a MSSQL',
        error: err.message
    });
}
});

// Iniciar servidor
const PORT = 3000;
app.listen(PORT, '0.0.0.0', () => {
    console.log(`Servidor API corriendo en http://0.0.0.0:${PORT}`);
    console.log('Esperando conexiones de ESP32...');
});

```

Funcionamiento del Servidor Node.js

El servidor recibe la información enviada por la ESP32.

Proceso:

1. Recibe score e id_rfid.
2. Consulta SQL Server.
3. Busca coincidencias por ID_RFID.

Si encuentra un registro:

UPDATE

- Suma la nueva puntuación.
- Actualiza LAST_GAME.

Si no encuentra registro:

INSERT

- Crea un nuevo jugador.
- Inicializa la puntuación.

Código del Arduino Ide

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#include <Adafruit_PN532.h>
```

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
#include <ArduinoJson.h>
```

```
// --- CONFIGURACIÓN PANTALLA OLED ---
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

```
// --- CONFIGURACIÓN NFC PN532 (I2C) ---
```

```
#define PN532_IRQ (2)
```

```
#define PN532_RESET (3) // No siempre es necesario conectarlo en I2C
```

```
Adafruit_PN532 nfc(PN532_IRQ, PN532_RESET);
```

```
// --- PINES DE ENTRADA ---
```

```
const int btnAzul = 12; // Continuar
```

```
const int btnRojo = 14; // Parar/No continuar
```

```
const int cables[4] = {15, 4, 16, 17}; // Pines de los cables de la bomba
```

```
// --- VARIABLES DE PUNTUACIÓN (MODIFICABLES) ---
```

```
int puntos_base = 100;
```

```
int mult_nivel2 = 3;
```

```
int mult_nivel3 = 4;
```

```
int puntos_actuales = 0;
```

```
// WIFI
```

```
const char* ssid = "TP-Link_3262";
```

```
const char* password = "99428167";
```

```
// NODE.JS
```

```
const char* scoreUrl =
```

```
"http://192.168.0.104:3000/api/puntuaciones";
```

```
// const char* scoreUrl = "http://192.168.0.104:3000/api/score";
```

```
// UID de la tarjeta actual
```

```
String id_rfid = "";
```

```
// Estados del juego

enum EstadoJuego { ESPERANDO_NFC, NIVEL_1, NIVEL_2, NIVEL_3,
PREGUNTA_CONT, FIN_JUEGO };

EstadoJuego estadoActual = ESPERANDO_NFC;

int nivelSuperado = 0;

void connectToWiFi() {

    Serial.println("Conectando WiFi...");

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("");
    Serial.println("WiFi conectado");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
}

void enviarScoreABD() {
```

```
if (WiFi.status() != WL_CONNECTED) {
```

```
    Serial.println("WiFi desconectado");
```

```
    return;
```

```
}
```

```
HTTPClient http;
```

```
http.begin(scoreUrl);
```

```
http.addHeader("Content-Type", "application/json");
```

```
DynamicJsonDocument doc(256);
```

```
// doc["score"] = puntos_actuales;
```

```
// doc["id_rfid"] = id_rfid;
```

```
doc["name_disp"] = "Inalambricas";
```

```
doc["usr"] = "Jugador";
```

```
doc["score"] = puntos_actuales;
```

```
doc["id_rfid"] = id_rfid;
```

```
String json;
```

```
serializeJson(doc, json);
```

```
Serial.println("Enviando:");
```

```
Serial.println(json);
```

```
int httpCode = http.POST(json);
```

```
Serial.print("HTTP CODE: ");
```

```
Serial.println(httpCode);
```

```
String respuesta = http.getString();
```

```
Serial.println("Respuesta servidor:");
```

```
Serial.println(respuesta);
```

```
http.end();
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    connectToWiFi();
```

```
    // Configurar botones y cables con Pull-Up interno
```

```
    pinMode(btnAzul, INPUT_PULLUP);
```

```
    pinMode(btnRojo, INPUT_PULLUP);
```

```
    for (int i = 0; i < 4; i++) {
```

```
        pinMode(cables[i], INPUT_PULLUP);
```

```
    }
```



```

// Inicializar OLED

if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("Fallo al iniciar SSD1306"));
    for (;;)
}

display.clearDisplay();
display.setTextColor(WHITE);


// Inicializar NFC

nfc.begin();

uint32_t versiondata = nfc.getFirmwareVersion();

if (!versiondata) {
    Serial.println("No se encontro la placa PN53x");
    while (1); // Detener
}

nfc.SAMConfig(); // Configurar para lectura


// Semilla para los números aleatorios

randomSeed(analogRead(0));


mostrarMensaje("Pasa tu\ntarjeta NFC\npara iniciar");
}


void loop() {
    switch (estadoActual) {

```

```

case ESPERANDO_NFC:
    esperarTarjeta();
    break;
case NIVEL_1:
    jugarNivel(1, 2); // Nivel 1, 2 cables
    break;
case NIVEL_2:
    jugarNivel(2, 3); // Nivel 2, 3 cables
    break;
case NIVEL_3:
    jugarNivel(3, 4); // Nivel 3, 4 cables
    break;
case PREGUNTA_CONT:
    preguntarContinuar();
    break;
case FIN_JUEGO:
    mostrarResultados();
    break;
}
}

// --- FUNCIONES DEL JUEGO ---

void esperarTarjeta() {
    uint8_t success;

    uint8_t uid[] = { 0, 0, 0, 0, 0, 0, 0, 0 }; // Buffer para el ID

```

```
uint8_t uidLength;           // Longitud del ID

    success = nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &uidLength,
200);

if (success) {

    id_rfid = "";

    Serial.print("ID de tarjeta detectado: ");

    for (uint8_t i = 0; i < uidLength; i++) {

        if (uid[i] < 0x10) {
            id_rfid += "0";
        }

        id_rfid += String(uid[i], HEX);

        Serial.print(uid[i], HEX);
    }

    id_rfid.toUpperCase();

    Serial.println("");
```

```
Serial.print("ID RFID guardado: ");
```

```
Serial.println(id_rfid);
```

```
// Asegurarse de que todos los cables estén conectados antes de empezar
```

```
if(verificarCablesConectados(2)) {
```

```
    puntos_actuales = 0;
```

```
    nivelSuperado = 0;
```

```
    estadoActual = NIVEL_1;
```

```
} else {
```

```
    mostrarMensaje("Conecta los\ncables primero!");
```

```
    delay(2000);
```

```
    mostrarMensaje("Pasa tu\ntarjeta NFC\npara iniciar");
```

```
}
```

```
}
```

```
}
```

```
bool verificarCablesConectados(int numCables) {
```

```
    for(int i=0; i<numCables; i++){
```

```
        if(digitalRead(cables[i]) == HIGH) return false; // Si está en HIGH, está  
        desconectado
```

```
    }
```

```
    return true;
```

```
}
```

```
void jugarNivel(int nivel, int numCables) {
```

```
    display.clearDisplay();
```

```
display.setCursor(0, 0);  
display.setTextSize(1);  
display.print("Nivel "); display.print(nivel); display.println(" elige un cable");  
display.display();
```

```
int cableCorrecto = random(0, numCables);
```

```
unsigned long tiempoInicio = millis();
```

```
bool cableCortado = false;
```

```
bool victoria = false;
```

```
// Bucle del temporizador de 5 segundos
```

```
while (millis() - tiempoInicio < 5000) {
```

```
    int tiempoRestante = 5 - ((millis() - tiempoInicio) / 1000);
```

```
    // Actualizar reloj en pantalla
```

```
    display.fillRect(0, 20, 128, 44, BLACK); // Limpiar zona inferior
```

```
    display.setCursor(0, 20);
```

```
    display.setTextSize(2);
```

```
    display.print("Tiempo: "); display.println(tiempoRestante);
```

```
    display.display();
```

```
// Comprobar si se cortó algún cable
```

```
for (int i = 0; i < numCables; i++) {
```

```
    if (digitalRead(cables[i]) == HIGH) { // Cable desconectado
```

```
        cableCortado = true;
```

```
        if (i == cableCorrecto) {
```

```

        victoria = true;
    }

    break; // Salir del for
}
}

if (cableCortado) break; // Salir del while si ya cortó uno
delay(100); // Pequeña pausa para no saturar la pantalla
}

// Evaluar resultado
if (victoria) {
    if(nivel == 1) puntos_actuales = puntos_base;
    else if(nivel == 2) puntos_actuales *= mult_nivel2;
    else if(nivel == 3) puntos_actuales *= mult_nivel3;

    nivelSuperado = nivel;
    mostrarMensaje("CORRECTO!");
    delay(2000);

    if (nivel == 3) {
        estadoActual = FIN_JUEGO; // Si gana el último nivel, termina
    } else {
        estadoActual = PREGUNTA_CONT;
    }
} else {

```

```

// Pierde (cortó el malo o se acabó el tiempo)

mostrarMensaje("BOOM! Perdiste");

delay(2000);

puntos_actuales = puntos_actuales / 2; // Penalización por perder

estadoActual = FIN_JUEGO;

}

}

void preguntarContinuar() {
    display.clearDisplay();
    display.setCursor(0, 0);
    display.setTextSize(1);
    display.println("Desea continuar?");
    display.println("");
    display.print("Puntos: "); display.println(""); display.println(puntos_actuales);
    display.println("");
    display.println("Azul: Continuar");
    display.println("Rojo: Retirarse");
    display.display();

    while (true) {
        if (digitalRead(btnAzul) == LOW) {
            // Avanzar al siguiente nivel

            if (nivelSuperado == 1) estadoActual = NIVEL_2;
            if (nivelSuperado == 2) estadoActual = NIVEL_3;

```

```
// Esperar a que suelte el botón y conecte los cables
while(digitalRead(btnAzul) == LOW);
mostrarMensaje("Reconecta cables...");
delay(2000);
break;
}
if (digitalRead(btnRojo) == LOW) {
    estadoActual = FIN_JUEGO;
    while(digitalRead(btnRojo) == LOW);
    break;
}
delay(50);
}
}
```

```
void mostrarResultados() {
    display.clearDisplay();
    display.setCursor(0, 0);
    display.setTextSize(2);
    display.println("FIN JUEGO");
    display.setTextSize(1);
    display.println("");
    display.print("Puntos finales: ");
    display.setTextSize(2);
    display.println("");
    display.println(puntos_actuales);
}
```



```

display.display();

Serial.print("Juego terminado. Puntos a base de datos: ");
Serial.println(puntos_actuales);
enviarScoreABD();

delay(5000); // Mostrar resultado por 5 segundos
estadoActual = ESPERANDO_NFC; // Reiniciar
mostrarMensaje("Pasa tu\ntarjeta NFC\npara iniciar");
}

void mostrarMensaje(String msg) {
    display.clearDisplay();
    display.setCursor(0, 0);
    display.setTextSize(1);
    display.println(msg);
    display.display();
}

```

Método de Conexión

Conexión WiFi

La ESP32 utiliza:

```
WiFi.begin(ssid,password);
```

para conectarse a la red inalámbrica.

Comunicación HTTP

Se utiliza la librería:

HTTPClient

para realizar peticiones POST al servidor.

Comunicación con Base de Datos

Node.js utiliza:

mssql

para establecer la conexión con SQL Server.

Posibles Problemas Encontrados

- **Error de conexión a SQL**

Problema:

Failed to connect to 192.168.x.x:1433

Causa:

IP incorrecta del servidor SQL.

Solución:

Verificar la IP mediante ipconfig.

- **Error HTTP -1**

Problema:

HTTP CODE: -1

Causa:

Servidor Node.js apagado.

Solución:

Ejecutar:

node servidor_api_esp32.js en powerShell

- **Error HTTP -11**

Problema:

HTTP CODE: -11

Causa:

Problemas de red o URL incorrecta.

Solución:

Verificar IP y endpoint.

- **Error "Cannot POST /api/score"**

Problema:

Ruta inexistente.

Causa:

La API utilizaba:

/api/puntuaciones

y la ESP enviaba datos a:

/api/score

Solución:

Modificar la URL utilizada por la ESP32.

- **Lectura incorrecta del RFID**

Problema:

UID diferente al esperado.

Causa:

Conversión incorrecta de bytes a hexadecimal.

Solución:

Agregar ceros a la izquierda cuando sea necesario.

Métodos de Aseguramiento

Para garantizar el correcto funcionamiento se realizaron las siguientes pruebas:

Prueba de conexión WiFi

Verificación mediante monitor serial.

Prueba de API

Uso de PowerShell:

Invoke-RestMethod

para validar respuestas HTTP.

Prueba de Base de Datos

Consultas SELECT para verificar inserciones.

Prueba de actualización

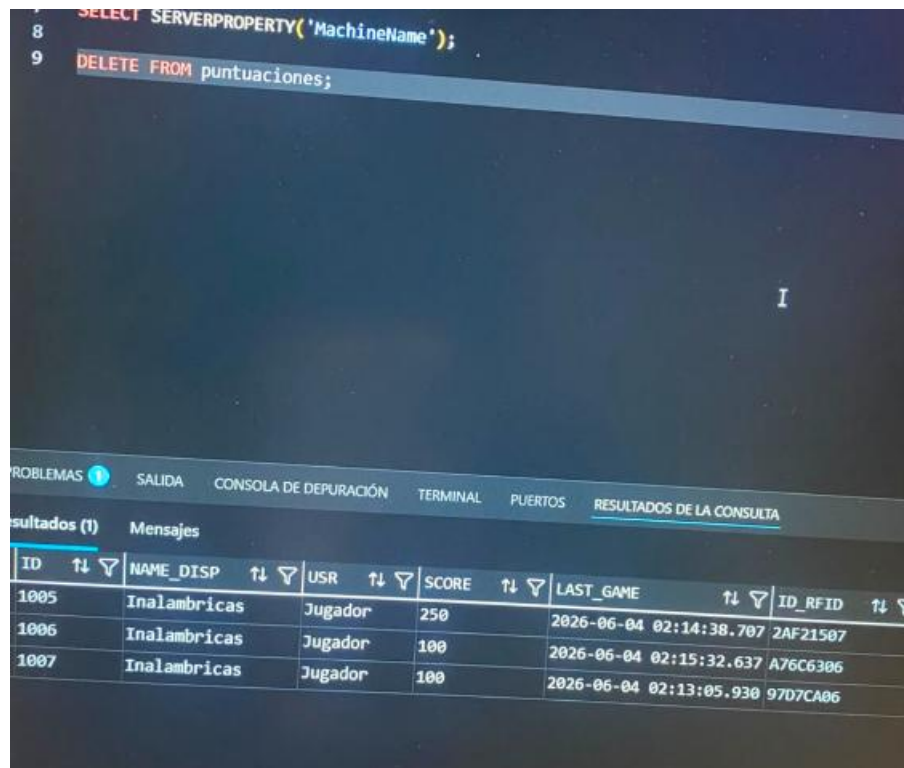
Verificación de acumulación correcta de puntos.

Prueba de interoperabilidad

Validación utilizando servidores y juegos de otros equipos.

Captura de la base de datos:

Podemos apreciar como se guardaron los datos esperado



The screenshot shows a SQL Server Enterprise Manager interface. At the top, a query window displays two SQL statements: `SELECT SERVERPROPERTY('MachineName');` and `DELETE FROM puntuaciones;`. Below the query window, the 'RESULTADOS DE LA CONSULTA' (Query Results) tab is active, showing a table with 7 columns: ID, NAME_DISP, USR, SCORE, LAST_GAME, and ID_RFID. The table contains three rows of data, all with a NAME_DISP of 'Inalambricas' and a USR of 'Jugador'.

ID	NAME_DISP	USR	SCORE	LAST_GAME	ID_RFID
1005	Inalambricas	Jugador	250	2026-06-04 02:14:38.707	2AF21507
1006	Inalambricas	Jugador	100	2026-06-04 02:15:32.637	A76C6306
1007	Inalambricas	Jugador	100	2026-06-04 02:13:05.930	97D7CA06

Conclusión

Manuel Humberto Reyes Salas: Durante el desarrollo se aplicaron conocimientos de programación, redes y bases de datos para crear un sistema arcade conectado. Además, se resolvieron distintos problemas de comunicación y configuración, fortaleciendo las habilidades prácticas

María José Guerrero Rios: Este proyecto permitió integrar tecnologías como ESP32, NFC, WiFi, Node.js y SQL Server en un sistema funcional. Se logró registrar jugadores mediante tarjetas RFID y almacenar sus puntuaciones de forma automática en una base de datos centralizada

Jesús Eli Sánchez Ruvalcaba: La práctica demostró cómo los dispositivos IoT pueden interactuar con servidores y bases de datos para automatizar procesos. El resultado fue un sistema capaz de identificar jugadores, guardar sus puntuaciones y mantener un historial de manera eficiente.

Link de video:

https://drive.google.com/file/d/1e0yw1Jqv0GWI3qoeFL3YQFw_NOGfg-a6/view

Link de repositorio:

https://github.com/atarisama/EvaluativoFinal_JuegoBomba